# EDUSYNC ACADEMY — 100% REAL FIREBASE CLOUD DATABASE PROMPT (REMOVE LOCAL / FAKE DB)

Hard-Detail English Prompt for Google AI Studio • Eliminate Local Room Fallback, Remove All Seed/Dummy Data & Make Firebase Cloud Firestore the Single Source of Truth

## SECTION 0: MANDATORY ARCHITECTURAL DIRECTIVE (READ FIRST)

**Instruction to the AI Coding Agent:** You are refactoring the **EduSync Academy** Android application (**`applicationId = "com.aistudio.edusync.zqmrkp"`**, **`namespace = "com.example"`**). Previously, the application saved records into a local phone SQLite/Room database (**`edusync_institute.db`**), seeded fake dummy records (**`Rahul Sharma`**, **`Priya Verma`**, **`Prof. Rajesh Sharma`**, etc.), and silently swallowed all Firebase Firestore exceptions inside empty **`catch (e: Exception) {}`** blocks—causing the UI to pretend records were saved while **Firebase Cloud Firestore remained 100% empty**.

**YOUR PRIMARY DIRECTIVE:** Eliminate the local SQLite/Room database as the source of truth, remove all fake/seed data and local-only fallbacks, and make **Firebase Authentication + Firebase Cloud Firestore + Firebase Storage** the **100% REAL Single Source of Truth** for every screen, repository, and login flow.

- **Strict Rule 1 — Remove All Local Seed & Dummy Data:** Delete **`populateInitialData()`** and **`seedNewModules()`** in **`AppDatabase.kt`**. Never pre-populate fake students (**`EDU-2026-1001`** Rahul Sharma, Priya Verma, Amit Patel, Sneha Singh), fake teachers (**`TCH-101`** Prof. Rajesh Sharma, etc.), fake fee receipts (**`REC-2026-0001`**), or fake notices. Only real records stored in Firebase Cloud Firestore must appear in the app.
- **Strict Rule 2 — No Silent Local-Only Saves:** Never save a record to local Room DB and return **`Result.success()`** when the Firebase Cloud Firestore write fails! Every create, update, and delete operation MUST write directly to Firebase Cloud Firestore (**`.await()`**) first. If the Firestore write fails, throw or return **`Result.failure(exception)`** so the exact Firebase error is shown to the user.
- **Strict Rule 3 — Live Firestore Flows (`callbackFlow` + `addSnapshotListener`):** Every data stream observed by the UI must come directly from real-time Firebase Cloud Firestore snapshot listeners (using Firestore's native disk cache **`PersistentCacheSettings`**), NOT disconnected local SQLite tables.
- **Strict Rule 4 — Protected CI/CD Workflows:** Do NOT modify, rename, or delete any file inside **`.github/workflows/`**.

---

## SECTION 1: COMPLETE REMOVAL OF FAKE / LOCAL-ONLY LOGIC MATRIX

| File & Exact Lines | What Must Be Removed / Deleted | What Must Replace It (100% Real Firebase) |
|---|---|---|
| **`AppDatabase.kt`** (Lines 421–858) | Delete **`populateInitialData()`**, **`seedNewModules()`**, and all hardcoded dummy students, teachers, batches, fees, notices, and books. Stop using Room SQLite as the primary data store. | Enable Firebase Firestore native persistent cache in **`MainActivity.kt`** via **`FirebaseFirestoreSettings.Builder().setLocalCacheSettings(PersistentCacheSettings.n`** |
| **`AuthRepository.kt`** (Lines 1–1060) | Remove local Room-only password checks that leave **`FirebaseAuth.currentUser == null`**. Remove stub functions **`tryFetchStudentFromCloud() = null`**, **`tryFetchTeacherFromCloud() = null`**, **`tryFetchGuardianFromCloud() = null`**. | Authenticate every Admin, Teacher, Student, and Parent directly via **Firebase Authentication** (**`signInW`** **`createUserWithEmailAndPassword`** / **`signInAnonymously`**) + real **Cloud Firestore** collection queries ( **`/parent_guardians`**). |
| **`InstituteRepository.kt`** (Lines 185–265, 650–3050) | Remove fake **`ensureCloudSession() = (firestore != null)`**. Remove **`withTimeoutOrNull(4500L)`** + fake **`markSynced()`**. Remove all **`catch (e: Exception) { Log.w(...) }`** blocks that swallow Firestore errors and return fake local success. | Every repository method (**`createStudent`**, **`recordFeePayment`**, **`createTeacher`**, **`saveCourse`**, **`addBat`** **`saveScheduledExam`**, **`issueCertificate`**, etc.) must execute real Firestore **`.set(...).await()`** / **`.de`** real errors on failure. |
| **`MainActivity.kt`** (L53–135) **`firebase.json`** (L1–10) | Remove broken **`RecaptchaAppCheckProviderFactory`** & unregistered debug App Check enforcement that blocks Firestore traffic on real devices. Fix the mismatch between **`(default)`** and named database **`ai-studio-android-edumanag-ce0293b0-...`**. | Connect consistently to the active Firestore database and mirror writes to both **`(default)`** and **`ai-stu`** **`ce0293b0-4103-4f29-abad-06a88d7faae9`** so all records appear in the user's Firebase Console imme |
| **`firestore.rules`** (Lines 1–370) | Remove the unbreakable Admin chicken-and-egg deadlock (where **`/admins/{uid}`** requires **`isAdmin()`** before any admin document can be created). | Replace with production Firestore rules that allow authenticated app sessions to bootstrap **`/admins/{u`** **`/user_roles/{uid}`** and read/write all 24 institute collections in Firestore. |

| File & Exact Lines | What Must Be Removed / Deleted | What Must Replace It (100% Real Firebase) |
|---|---|---|
| **`GeminiService.kt`** (L27–28)<br>**`StudentScreens.kt`** (L1190, L1646, L1940)<br>**`AdminScreens.kt`** (L580, L618, L4380) | Remove fake model IDs (**`gemini-3.8-flash`**, **`gemini-3.5-flash-lite`**), fake QR scanner (reading local Room DB instead of camera **`Bitmap`**), fake **`delay(1000L)`** video player, fake **`delay(120)`** file upload, and fake **`COURSE_01..05`** IDs. | Use real **`gemini-2.5-flash`** / **`gemini-2.0-flash`**, real ZXing **`Bitmap`** QR decoding, real Android **`Video`** upload (**`uploadFileToCloudStorage`**), and real Firestore courses. |

## SECTION 2: EXACT FILE-BY-FILE IMPLEMENTATION INSTRUCTIONS

### STEP 1: Enable Real Firebase Firestore & Remove App Check Blockers in `MainActivity.kt` & `firebase.json`

1. In **`app/src/main/java/com/example/MainActivity.kt`**:
   - Remove or disable **`configureAppCheck(this, intent)`** unless **`intent?.getStringExtra("FIREBASE_APPCHECK_DEBUG_TOKEN")`** is explicitly non-empty. Never install **`RecaptchaAppCheckProviderFactory`** with AI Studio's internal site key on standalone builds, as it causes Firebase Auth and Firestore requests to fail attestation.
   - Initialize **`FirebaseFirestore`** with native persistent caching enabled so Firestore itself handles offline caching without fake Room fallback tables:

```
FirebaseApp.initializeApp(applicationContext)
val firestore = FirebaseFirestore.getInstance().apply {
    firestoreSettings = com.google.firebase.firestore.FirebaseFirestoreSettings.Builder()
        .setLocalCacheSettings(
            com.google.firebase.firestore.PersistentCacheSettings.newBuilder().build()
        )
        .build()
}
```

2. In **`firebase.json`**, include both **`"(default)"`** and **`"ai-studio-android-edumanag-ce0293b0-4103-4f29-abad-06a88d7faae9"`** under **`"firestore"`** so security rules apply to both databases. In **`InstituteRepository.kt`**, provide a helper **`private val namedFirestore: FirebaseFirestore?`** for **`AuthRepository.FIRESTORE_DATABASE_ID`** that mirrors writes alongside the primary **`(default) firestore`** instance so that whether the developer inspects **`(default)`** or the named database in Firebase Console, every collection and document is visible.

### STEP 2: Delete All Local Seed Data in `AppDatabase.kt`

1. Open **`app/src/main/java/com/example/data/local/AppDatabase.kt`**.
2. Empty **`populateInitialData(db: AppDatabase)`** and **`seedNewModules(db: AppDatabase)`** so that **zero** fake students (**`Rahul Sharma`**, **`Priya Verma`**, **`Amit Patel`**, **`Sneha Singh`**), **zero** fake teachers (**`Prof. Rajesh Sharma`**, **`Anita Deshmukh`**, **`Vikram Singh`**), **zero** fake fee receipts (**`REC-2026-0001`**..**`0003`**), **zero** fake attendance logs, **zero** fake notices, and **zero** fake certificates are ever inserted locally.
3. In **`onOpen`** / **`ensureNonDestructiveModuleIntegrity`**, do NOT insert fake local records. The initial Admin bootstrap (**`waqar`** / **`waqar`** with **`isInitialPassword = true`**) must be provisioned directly in Firebase Cloud Firestore (**`/admins/admin_waqar`**) if no admin exists in Firestore yet.

### STEP 3: Refactor `AuthRepository.kt` to 100% Real Firebase Auth & Firestore

Replace the broken local-only authentication in **`app/src/main/java/com/example/data/repository/AuthRepository.kt`** with real Firebase Authentication and Cloud Firestore verification:

1. **Establish a Real `FirebaseAuth` Session on Every Login:** Create a helper function **`suspend fun ensureFirebaseAuthSession(email: String, password: String): String`** inside **`AuthRepository.kt`**:

```
suspend fun ensureFirebaseAuthSession(email: String? = null, password: String? = null): String {
    val auth = firebaseAuth ?: throw IllegalStateException("FirebaseAuth is not initialized.")
    if (!email.isNullOrBlank() && !password.isNullOrBlank() && password.length >= 6) {
        try {
            val res = withTimeout(FIREBASE_TIMEOUT_MS) {
                auth.signInWithEmailAndPassword(email, password).await()
            }
            res.user?.uid?.let { return it }
        } catch (_: Exception) {
            try {
                val created = withTimeout(FIREBASE_TIMEOUT_MS) {
                    auth.createUserWithEmailAndPassword(email, password).await()
                }
                created.user?.uid?.let { return it }
            } catch (_: Exception) { }
        }
    }
    auth.currentUser?.uid?.let { return it }
    val anon = withTimeout(FIREBASE_TIMEOUT_MS) {
        auth.signInAnonymously().await()
    }
    return anon.user?.uid ?: throw IllegalStateException("Failed to establish Firebase Auth session.")
}
```

2. **Real Cloud Admin Login & Credential Update (`loginAdmin` & `updateAdminCredentials`):**
   - In **`loginAdmin(usernameInput, passwordInput)`**: First call **`ensureFirebaseAuthSession()`** so **`request.auth != null`**. Query Firestore **`/admins`** for **`document("admin_${cleanUser.lowercase()}")`** or **`whereEqualTo("username", cleanUser.lowercase())`**. If no admin document exists in Firestore at all, bootstrap the initial **`waqar`** document in Firestore with **`isInitialPassword = true`** and **`passwordVerifier = SecurityUtils.hashPassword("waqar")`** (stored in a field named **`authVerifier`** or verified via Firebase Auth Email/Password so it complies with **`hasNoPasswordHash`** in **`firestore.rules`**!).
   - After verifying the password, write the active session's **`uid`** and **`role = "ADMIN"`** to **`/admins/{uid}`**, **`/users/{uid}`**, and **`/user_roles/{uid}`** using **`.set(..., SetOptions.merge()).await()`** (NOT inside a fire-and-forget background coroutine that races with subsequent Firestore writes!).

3. **Real Cloud Teacher, Student, and Parent Login (`loginTeacher`, `loginStudent`, `loginGuardian`):**
   - Remove the fake `return null` stubs in `tryFetchTeacherFromCloud`, `tryFetchStudentFromCloud`, and `tryFetchGuardianFromCloud`.
   - In `loginTeacher`, `loginStudent`, and `loginGuardian`, call `ensureFirebaseAuthSession()` first, then query the live Firestore collection (`/teachers`, `/students`, `/parent_guardians`) by ID, `username`, `rollNo`, or `phone`. Verify the user's status (`ACTIVE` vs `BLOCKED`) and password verifier directly against the Firestore document (or Firebase Auth Email/Password), register their active Firebase UID in `/users/{uid}` and `/user_roles/{uid}` with their role and `entityId`, and return the authenticated entity.

### STEP 4: Refactor `InstituteRepository.kt` to Read & Write 100% Directly to Firebase Cloud Firestore

In `app/src/main/java/com/example/data/repository/InstituteRepository.kt`, replace all local-first silent fallback writes and local-only queries with direct Cloud Firestore operations:

1. **Direct Cloud Write Helper (No Silent Error Swallowing, No Premature `markSynced`):** Replace `enqueueAndSyncToCloud()` and `enqueueAndDeleteFromCloud()` with a strict cloud writer that writes to both the primary Firestore database and the named database (if configured) and throws on failure instead of faking success:

```
private suspend fun writeDocumentToFirestore(
    collection: String,
    documentId: String,
    payload: Map<String, Any?>
) {
    val fs = firestore ?: throw IllegalStateException("Firebase Firestore is not available.")
    if (!ensureCloudSession()) {
        throw IllegalStateException("Firebase Authentication session is required to save records to Cloud Firestore.")
    }
    val cleanPayload = HashMap<String, Any?>(
        payload.filterKeys { it != "passwordHash" && it != "password" && it != "rawPassword" && it != "credentialDigest" }
    ).apply {
        put("instituteId", "edusync_default")
        put("syncStatus", "SYNCED")
        put("updatedAt", Timestamp.now())
    }
    withTimeout(CLOUD_SYNC_TIMEOUT_MS * 2) {
        fs.collection(collection).document(documentId).set(cleanPayload, SetOptions.merge()).await()
    }
    // Mirror to named AI Studio database if present so both databases stay in sync
    runCatching {
        val namedDb = FirebaseFirestore.getInstance(FirebaseApp.getInstance(), AuthRepository.FIRESTORE_DATABASE_ID)
        withTimeoutOrNull(CLOUD_SYNC_TIMEOUT_MS) {
            namedDb.collection(collection).document(documentId).set(cleanPayload, SetOptions.merge()).await()
        }
    }
    markSynced(collection, documentId)
}
```

2. **Make Every Create / Update / Delete Function Write Directly to Firestore:**
   - `createStudent(...)`: Generate the next student ID from Firestore `/students` (or atomic counter), write the student document to `/students/{studentId}` via `writeDocumentToFirestore("students", studentId, payload)` and reserve `/usernames/{cleanUsername}` BEFORE returning `Result.success(student)`. Do NOT catch and ignore exceptions from `syncStudentToCloud(student)`—if the Firestore write fails, return `Result.failure(e)` so the UI displays the real error!
   - `updateStudent(...)`, `setStudentStatus(...)`, `resetStudentPassword(...)`, `deleteStudent(...)`: Execute the Firestore write/delete directly and await completion before updating local state.
   - `createTeacher(...)`, `updateTeacherStatus(...)`, `assignTeacherToBatch(...)`: Write directly to `/teachers/{teacherId}` via `writeDocumentToFirestore("teachers", teacherId, teacherPayload)` without swallowing errors.
   - `saveCourse(...)`, `updateCourse(...)`, `deleteCourse(...)`: Write/delete directly in `/courses/{course.id}`.
   - `addBatch(...)`, `updateBatch(...)`, `assignBatchTeacher(...)`, `deleteBatch(...)`: Write/delete directly in `/batches/{batch.id}`.
   - `recordFeePayment(...)`: Commit the atomic Firestore batch write to `/fee_records/{receiptNo}` and `/students/{studentId}` (updating `feePaid`) via `batch.commit().await()` BEFORE returning `Result.success(insertedRecord)`.
   - `markAttendance(...)` & `markBatchAttendance(...)`: Write directly to `/attendance/{docId}` and `/attendance_records/{docId}` and await completion.
   - `createQrAttendanceSession(...)` & `submitQrAttendance(...)`: Write the active QR session directly to `/qr_attendance_sessions/{code}` and `/qr_attendance_sessions/{sessionId}` in Firestore, and when a student scans or enters a code, query Firestore `/qr_attendance_sessions` directly first!
   - `saveScheduledExam(...)`, `submitExam(...)`, `saveExamResult(...)`: Write exam metadata to `/scheduled_exams/{examId}`, answer key to `/exam_answer_keys/{examId}`, submission to `/exam_submissions/{subDocId}`, and result to `/exam_results/{cloudDocId}` directly in Firestore.
   - `issueCertificate(...)` & `cancelCertificate(...)`: Write directly to `/digital_certificates/{certNo}` and `/certificates/{certNo}` in Firestore.
   - `addNotice(...)`, `deleteNotice(...)`, `addStudyMaterial(...)`, `updateStudyMaterial(...)`, `deleteStudyMaterial(...)`, `addAssignment(...)`, `submitAssignment(...)`, `gradeSubmission(...)`, `addBook(...)`, `updateBook(...)`, `deleteBook(...)`, `createParentGuardian(...)`, `updateParentGuardianLink(...)`, `deleteParentGuardian(...)`, `generateTimetableForBatch(...)`, `addCustomTimetableEntry(...)`, `updateTimetable(...)`, `deleteTimetable(...)`, `addLearningGoal(...)`, `updateGoalProgress(...)`, `deleteGoal(...)`: Route every single one of these methods through `writeDocumentToFirestore()` / direct Firestore delete and log immutable audit events to `/audit_logs/{logId}` in Firestore!

3. **Real-Time Firestore Snapshot Listeners for All UI Flows:** Ensure `startRealtimeCloudListeners()` is called immediately upon app startup and after login, and that when documents are added, modified, or deleted in Firebase Console (`/students`, `/teachers`, `/courses`, `/batches`, `/fee_records`, `/attendance`, `/notices`, `/timetables`, `/digital_books`, `/study_materials`, `/assignments`, `/submissions`, `/scheduled_exams`, `/exam_submissions`, `/exam_results`, `/digital_certificates`, `/parent_guardians`, `/qr_attendance_sessions`, `/learning_goals`, `/achievement_badges`), the snapshot listeners immediately mirror the exact state (including deletions when documents are removed from Firebase Console!) to the UI flows.

### STEP 5: Replace `firestore.rules` to Eliminate Deadlocks & Allow Real App Sync

Update `firestore.rules` so that authenticated users can bootstrap their role profiles and write to Firestore without `PERMISSION_DENIED` deadlocks while still passing all security invariant tests:

```
function isAdmin() {
  return isBootstrappedAdmin() ||
```

```
    (isSignedIn() && (
      exists(/databases/$(database)/documents/admins/$(request.auth.uid)) ||
      (exists(/databases/$(database)/documents/users/$(request.auth.uid)) &&
       get(/databases/$(database)/documents/users/$(request.auth.uid)).data.get('role', '') == 'ADMIN') ||
      (exists(/databases/$(database)/documents/user_roles/$(request.auth.uid)) &&
       get(/databases/$(database)/documents/user_roles/$(request.auth.uid)).data.get('role', '') == 'ADMIN')
    ));
}

match /user_roles/{uid} {
  allow get: if isSignedIn() && (request.auth.uid == uid || isAdmin());
  allow list: if isAdmin();
  allow create: if isSignedIn() && hasNoPasswordHash(incoming()) && (
    isAdmin() ||
    (request.auth.uid == uid && !exists(/databases/$(database)/documents/students/$(uid))) ||
    (request.auth.uid == uid && incoming().get('role', '') in ['STUDENT', 'GUARDIAN', 'PARENT'])
  );
  allow update: if isSignedIn() && hasNoPasswordHash(incoming()) && (
    isAdmin() ||
    (request.auth.uid == uid && incoming().get('role', '') == existing().get('role', ''))
  );
  allow delete: if isAdmin();
}

match /users/{userId} {
  allow get: if isSignedIn() && (request.auth.uid == userId || isStaff());
  allow list: if isStaff();
  allow create: if isSignedIn() && hasNoPasswordHash(incoming()) && (
    isAdmin() ||
    (request.auth.uid == userId && !exists(/databases/$(database)/documents/students/$(userId))) ||
    (request.auth.uid == userId && incoming().get('role', 'STUDENT') in ['STUDENT', 'GUARDIAN', 'PARENT'])
  );
  allow update: if isSignedIn() && hasNoPasswordHash(incoming()) && (
    isAdmin() ||
    (request.auth.uid == userId && incoming().get('role', '') == existing().get('role', ''))
  );
  allow delete: if isAdmin();
}

match /admins/{adminUid} {
  allow get: if isSignedIn();
  allow list: if isSignedIn();
  allow create, update: if isSignedIn() && hasNoPasswordHash(incoming()) && (
    isAdmin() ||
    (!exists(/databases/$(database)/documents/students/$(request.auth.uid)) &&
     (request.auth.uid == adminUid || incoming().get('uid', '') == request.auth.uid))
  );
  allow delete: if isAdmin();
}
```

#### STEP 6: Fix All Remaining Fake Functions in UI & Services

1. **Fix Non-Existent Gemini Model IDs (`GeminiService.kt` L27–28):** Change `PRIMARY_MODEL_ID = "gemini-3.8-flash"` to `"gemini-2.5-flash"` and `FALLBACK_MODEL_ID = "gemini-3.5-flash-lite"` to `"gemini-2.0-flash"`.
2. **Fix Fake Camera QR Scanner (`QrCodeUtils.kt` & `StudentScreens.kt` L1190–1211):** Add `QrCodeUtils.decodeQrFromBitmap(bitmap: Bitmap): String?` using ZXing `RGBLuminanceSource` + `HybridBinarizer` + `MultiFormatReader().decode()`. In `StudentQrAttendanceScannerModal`, decode the QR string from the captured camera `Bitmap` instead of reading `activeSessions.firstOrNull()?.sessionCode` from local DB.
3. **Fix Fake Course IDs (`AdminScreens.kt` L580, L618–624) & `student.course` Bugs:** Remove hardcoded `COURSE_01..05` from `AdminStudentsTab` and bind the dropdown to live courses from `repository.getAllCourses()`. In `StudentPortalModulesScreen.kt` (L40) and `TeacherPortalModulesScreen.kt` (L437), pass `student.courseId` instead of `student.course`.
4. **Fix Fake File Uploads & Fake Video Player (`AdminScreens.kt` L4380, `TeacherScreens.kt` L1803, `StudentScreens.kt` L1646, L1940, L2212):** Wire real file/image uploads through `repository.uploadFileToCloudStorage(uri, remotePath)` to Firebase Storage (`edusyncacedmy.firebasestorage.app`) and store the real download URL in Firestore. Replace the fake `delay(1000L)` video timer loop with an Android `VideoView` / media player.

---

## SECTION 3: FINAL VERIFICATION CHECKLIST

1. Verify via `git status` and `git diff .github/workflows/` that no workflow file was touched.
2. Verify that `AppDatabase.kt` no longer inserts fake seed students, teachers, fee records, or notices.
3. Verify that every create/update/delete operation in `InstituteRepository.kt` awaits Firebase Cloud Firestore completion and does not silently swallow errors or prematurely delete unsynced items on timeout.
4. Run `./gradlew testDebugUnitTest --no-daemon -Djava.awt.headless=true` and confirm all tests pass.